하나금융티아이 CoP 결과공유회 · 프로젝트형

JARVIS

AIOps로 장애 대응 시간을 줄이다

로그 수집 → 탐지 → 진단 → AI 추천 → 운영자 승인 → 실행 → 복구 확인

Hanati Jarvis · 2026.07 – 2026.10 · hanati_jarvis master 브랜치 기준

CONTENTS

오늘의 이야기

01

BACKGROUND

배경

복구 속도가 경쟁력이 된 시대,
여전히 사람의 경험에 기대는
장애 대응과 CoP 목표

약 2분 30초

02

PROBLEM

문제

MTTR의 대부분은
‘조치’가 아니라 ‘판단’에서.
AI 도입 앞의 네 가지 벽

약 2분

03

SOLUTION

해결

JARVIS 파이프라인과
5가지 핵심 기능, 시연,
중간발표 이후의 진척

약 8분 · 시연 포함

04

SCALE-UP

스케일업

운영 데이터 학습과
단계적 자동화,
조직 전체로의 확산

약 2분 30초

서비스가 복잡해질수록, 복구 속도가 경쟁력입니다

운영 복잡도 증가

MSA · 클라우드 전환
장애 원인의 조합이 폭발적으로 확대

›

AIOps 전환

로그 · 메트릭을 AI로 분석
진단과 대응을 가속

›

핵심 SLA = MTTR

장애 발생 여부보다
복구까지 걸린 시간이 중요

24 / 7 무중단

결제 · 이체 · 인증은 몇 분만 멈춰도 거래 실패와 민원, 대외 신뢰 하락으로 이어집니다.

얽혀 있는 구성요소

API · DB · Redis · Kafka · 외부 PG · 인증 서버 · 컨테이너. 장애 하나가 여러 곳의 오류로 번집니다.

흩어진 신호

앱 로그, 호스트 자원, 과거 장애 기록이 각기 다른 곳에 있어 사람이 머릿속에서 이어 붙입니다.

핵심 변화 — '장애가 없는 서비스'보다 '가장 빨리 복구하는 서비스'가 경쟁력이 됩니다.

그런데 장애 대응은 아직 개인의 경험에 머물러 있습니다

1. 로그 확인
담당자가 많은 로그를
직접 눈으로 확인

›

2. 기억 · 위키 검색
과거 유사 장애를
개인의 기억에 의존해 탐색

›

3. 대응 결정
경험치에 따라
판단 속도와 품질이 달라짐

공통 원인 — 대응 지식이 시스템이 아니라 사람에게 축적

속도 · 품질 편차

숙련도와 담당자 부재 여부에 따라 대응 수준이 달라집니다. 야간 · 주말일수록 더 늦어집니다.

노하우 재사용 불가

과거 해결 이력이 검색 가능한 조직 자산으로 남지 않습니다. 같은 장애도 매번 처음부터 조사합니다.

그래서 우리 CoP는 네 가지 목표를 세웠습니다

GOAL 1

장애 대응 전 과정을 하나의 서비스로

수집 → 진단 → 추천 → 조치 → 복구 확인까지 끊김 없는 End-to-End 파이프라인

GOAL 2

과거 이력이 검색되는 진단 · 추천

금융 빈발 장애 20종 이상을 자동 분류하고, 과거 사례를 근거로 원인 · 조치를 추천

GOAL 3

운영자가 고르면 안전하게 실행

AI는 판단을 보조하고, 최종 실행 결정권은 운영자에게 남기는 신뢰 가능한 반자동화

GOAL 4

현업 적용 가능한 운영 수준

실서버 배포, 자동 검증, 민감정보 보호까지 갖춘 “바로 써볼 수 있는” 완성도

4개월	5명	99건	158개
2026.07 – 2026.10	코드 기여자	master 커밋	자동 검사 항목

MTTR의 대부분은 '조치'보다 '판단'에서 발생합니다

초기 대응(Triage) 병목

장애 탐지
알림 · 로그 발생

›

원인 파악
로그 조사 · 증거 확인

›

대응 결정
조치 방법 판단

사람이 로그를 훑고 판단하는 시간 = 전체 MTTR의 대부분

실제 조치
명령 · 스크립트 실행

실제 조치는
상대적으로 짧음

유사 장애도 매번 재조사
과거 해결 방법을 다시 찾는 데 시간 소모

담당자 부재 시 품질 급락
숙련자 의존 → 야간 · 주말 리스크

다운타임 · SLA 손실
판단이 늦어질수록 거래 손실 확대

온콜 인력 번아웃
반복 알림 · 반복 조사의 피로 누적

판단을 AI에 맡기려 할 때 마주치는 네 가지 벽

탐지

P1. 알림 폭주와 노이즈

장애 하나가 “read-only mode 전환”, “쓰기 실패” 같은 **후속 ERROR**를 줄줄이 남기고, 반복 오류까지 개별 장애로 쌓여 진짜 원인이 묻힙니다.

판단

P2. 근거 없는 AI 추천

과거 사례와 진단 결과 없이 LLM만 쓰면 **그럴듯하지만 틀린 원인**을 내놓습니다. 처음 보는 오류일수록 위험합니다.

조치

P3. 위험한 실행

엉뚱한 호스트 재시작, 같은 명령의 **중복 실행**, 이중화 없는 재시작은 장애를 키웁니다. AI가 바로 실행하면 책임 소재도 사라집니다.

확인 · 보안

P4. 복구 미검증 · 정보 유출

“명령 성공 = 복구”로 오판해 재발합니다. 로그 속 **주민번호 · 카드 · 계좌 · 토큰**이 외부 LLM으로 나갈 위험도 있습니다.

빠르게 하려면 위험하고, 안전하게 하려면 느립니다. 이 딜레마를 동시에 풀어야 합니다.

우리가 풀어야 할 질문

어떻게 하면 AI의 속도로 원인을 찾고, 사람의 통제 아래에서 검증된 복구까지 끝낼 수 있을까?

요구 조건	설계 원칙
빠르게	수집 즉시 비동기 분석, 반복 · 후속 오류는 묶어서 한 번만 분석
근거 있게	과거 사례 + 진단 결과 + 자원 지표를 근거로 제시하고, 확인되지 않은 원인은 단정하지 않음
안전하게	AI는 판단 보조, 실행 결정권은 운영자에게 — 대상 · 버전 · 만료 · 중복 실행은 서버가 검증
끝까지	명령 완료와 업무 복구 확인을 분리하고, 해결 사례를 다시 지식으로 축적

JARVIS 전체 파이프라인 — 문제 4개에 핵심 기능 5개로

수집
Fluent Bit
Collector

›

집약 · 분류
오류 21종
P1~P3

›

진단 · 검색
진단 스크립트
Qdrant + ES

›

AI 추천
LLM + 품질 게이트
자원 가설

›

운영자 승인
최종 결정권
사람

›

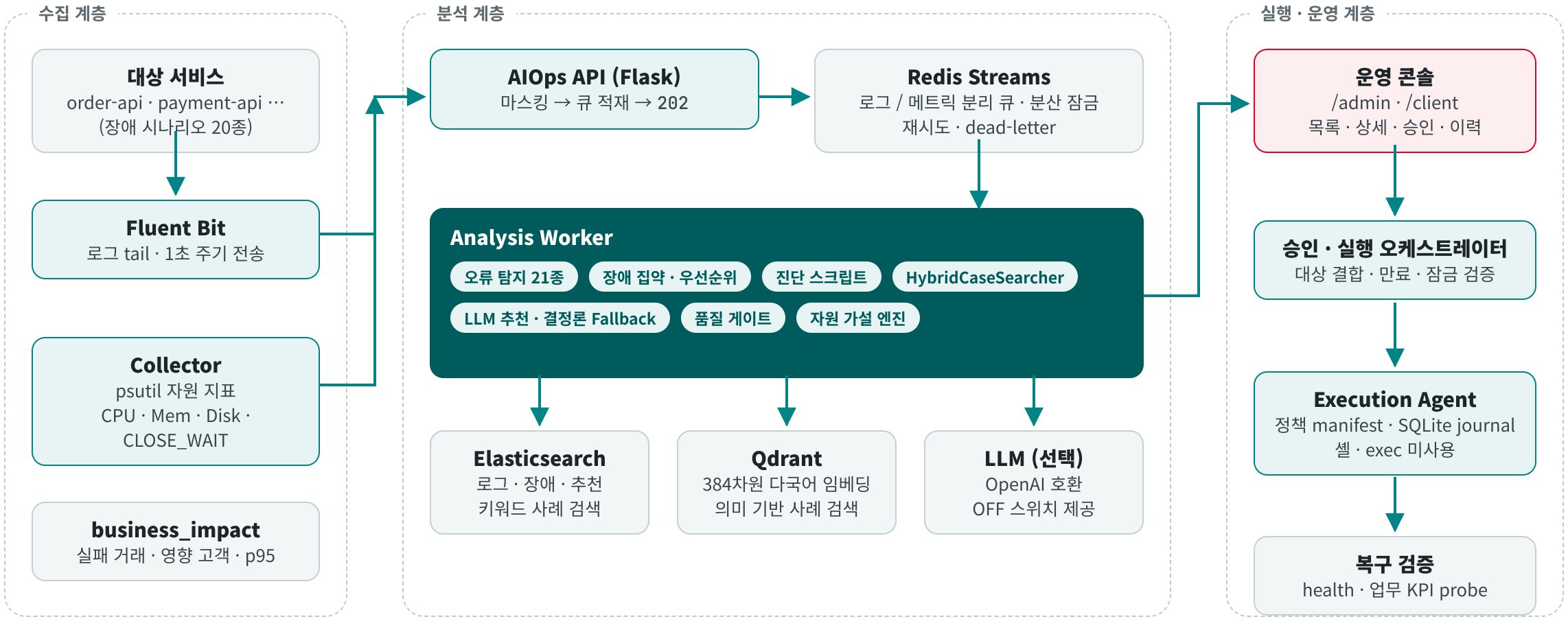
안전 실행
서버 검증
정책 Agent

›

복구 확인 · 학습
health + KPI
사례 축적

#	핵심 기능	해결하는 문제	한 줄 요약
F1	**지능형 장애 집약 · 업무 우선순위**	P1 알림 폭주	후속 · 반복 오류를 하나의 장애로 묶고 업무 영향으로 P1~P3 부여
F2	**하이브리드 원인 분석**	P2 근거 없는 추천	Qdrant + Elasticsearch 사례 검색과 진단 결과를 근거로 LLM이 추천, 품질 미달 시 자원 가설로 전환
F3	**승인 기반 안전 실행**	P3 위험한 실행	운영자 승인 + 서버의 8가지 조건 검증을 통과해야만 실행
F4	**복구 검증 · 피드백 학습**	P4 복구 미검증	명령 완료와 업무 복구를 분리 검증, 확정 사례를 지식으로 축적
F5	**금융권 보안 · 거버넌스**	P4 정보 유출	전 구간 민감정보 마스킹, 외부 LLM 차단 스위치, 감사 기록

Docker Compose 한 번으로 뜨는 이벤트 기반 AIOps 구조



로그 100줄이 아니라 장애 1건으로 보여줍니다

F1

202 — **비동기 수집** — API는 민감정보를 마스킹해 Redis Streams에 넣고 즉시 202 + 작업 ID 반환. 분석은 별도 Worker가 처리해 수집이 막히지 않습니다.

21종 — **오류 코드 자동 분류** — DISK_FULL, DB_DEADLOCK, CONNECTION_POOL_EXHAUSTED, PAYMENT_GATEWAY_FAILURE, ORA-28040 등 금융 시스템 빈발 장애를 패턴으로 식별합니다.

39개 — **후속 메시지 제외** — “Service entering read-only mode.” 같은 후속 ERROR는 원문 로그로만 보존하고 별도 장애를 만들지 않습니다.

300초 — **반복 분석 재사용** — 같은 서비스 · 메시지 · 오류 코드는 기존 장애에 발생 수만 누적. 유효한 추천이 있으면 LLM을 다시 부르지 않습니다.

업무 영향 기반 P1~P3

등급	판정 기준
P1	업무 중단 또는 긴급 기준 초과
P2	tier1 핵심 업무, 실패 거래 · 영향 고객 발생
P3	업무 영향 미확인

횟수만으로 업무 중단을 확정하지 않고, 보고된 피해 지표를 근거로만 등급을 올립니다.

콘솔은 **최근 10분** 장애를 진행중 · 긴급 · 미확인 · 분석중 탭으로 보여주고, 1분 이내 발생 건을 빨간 테두리로 강조합니다.

SOLUTION · F2 하이브리드 원인 분석

“과거 이력이 검색되는” 진단을 실제로 구현했습니다

F2

① 진단 실행
오류 코드별
진단 스크립트

›

② 사례 검색
Qdrant + Elasticsearch
Top-3 과거 장애

›

③ LLM 추천
원인 + 조치 Runbook
추천도 순 랭킹

›

④ 품질 게이트
후보 없음 · 빈 응답 ·
낮은 신뢰도 차단

›

⑤ 자원 가설 엔진
15분 지표 기반
가설 · 추가 진단

Qdrant — 의미 기반 검색

다국어 임베딩(384차원)으로 **표현이 달라도 뜻이 비슷한** 과거 사례를 찾습니다.

Elasticsearch — 키워드 검색

에러 코드 · 고유명사처럼 **정확히 일치해야 하는** 사례를 찾습니다. 두 결과를 합쳐 점수순으로 정렬하고, 한쪽이 장애여도 다른 쪽으로 계속 검색합니다.

LLM이 없어도 동작

API 키가 없거나 응답이 깨지면 **결정론적 랭커**가 규칙 순서 + 사례 유사도로 순위를 만듭니다.

모르면 모른다고 말하기

근거가 부족하면 원인을 지어내지 않고, CPU · 메모리 · 디스크 · CLOSE_WAIT 등 **지표 근거가 있는 가설**만 제시합니다.

운영 환경에서 자주 발생하는 장애 20종으로 검증했습니다

log_generator 시나리오를 관리자 화면에서 실행 → 오류 코드 탐지 → 진단 · 추천 Runbook 연결 (일부 발췌)

시나리오 (오류 코드)	자동 진단 스크립트	추천 조치 Runbook (우선순위 순)
디스크 부족 DISK_FULL	check_disk_usage, check_large_files	① compress_old_logs ② cleanup_temp_files
DB 커넥션 실패 DB_CONNECTION_FAILURE	check_db_connection	① restart_db_connection_pool ② failover_database
커넥션 풀 고갈 CONNECTION_POOL_EXHAUSTED	check_connection_pool	① increase_connection_pool_size ② restart_db_connection_pool
DB 데드락 DB_DEADLOCK	check_db_locks	① kill_blocking_session ② retry_failed_transactions
결제 게이트웨이 장애 PAYMENT_GATEWAY_FAILURE	check_payment_gateway	① switch_payment_provider ② enable_payment_retry_queue
Redis 연결 끊김 REDIS_CONNECTION_FAILURE	check_redis_status	① restart_redis ② clear_redis_cache
인증 토큰 검증 실패 AUTH_TOKEN_VALIDATION_FAILURE	check_auth_service_health	① rotate_signing_key ② restart_auth_service
SSL 인증서 만료 SSL_CERTIFICATE_EXPIRED	check_ssl_certificate	① renew_ssl_certificate ② reload_tls_config

20개 중 19개
시나리오에 Runbook 자동 연결

58개
진단 · 조치 스크립트 SHA-256 등록

1개 (Rate Limit)
대응 스크립트 없음 → 수동 조치 등록

SOLUTION · F3 승인 기반 안전 실행

“운영자가 고르면 실행되는” 단계, 구현을 완료했습니다

F3

대상 선택
추천에 연결된 환경 · 서비스 · 호스트

›

승인 후 실행
운영자 클릭 1회 (승인자 기록)

›

서버 검증
8가지 실행 조건 실패 시 사유 표시

›

명령 실행
Agent가 실행 직전 조건 재확인

›

MONITORING
복구 확인 전까지 해결로 보지 않음

실행 전 서버가 확인하는 조건
① 장애 · 추천 ID / 버전 / 유효 시간 일치
② 대상 환경 · 서비스 · 호스트 · 인스턴스 결합
③ Agent 인증 토큰 · 정책 해시
④ 승인된 Compose 프로젝트 · 컨테이너 · 이미지
⑤ 이중화 — 건강한 peer가 있어야 재시작 허용
⑥ 점검 라벨 · ⑦ 실행 전 상태 복원 가능성
⑧ Redis 분산 잠금 — 동시 · 중복 실행 차단

사고를 만들지 않는 실행

- **셸 · 호스트 명령 · exec 미사용** — Docker API 내장 작업만 허용
- **SQLite journal** — 응답 유실 시 재실행 없이 원래 기록 조회
- 재시작 실패 시 **실행 전 상태로 자동 복원**
- Runbook은 **SHA-256**으로 등록, 파일 변경 시 실행 차단

AI는 판단을 보조하고, **최종 결정권은 운영자**에게. 대응 스크립트가 없으면 **수동 조치 등록**으로 이력을 남깁니다.

SOLUTION · F4 복구 검증 · 피드백 학습 루프

“재시작 성공”이 아니라 “업무 정상화”를 확인합니다

F4

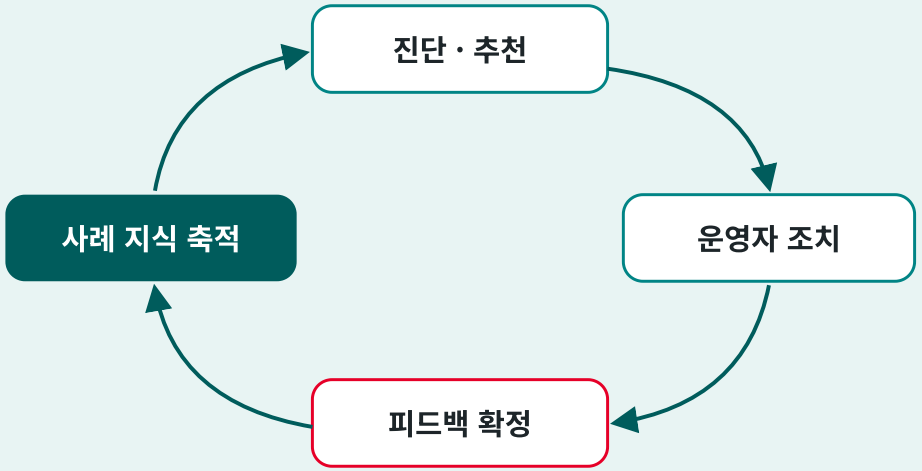
2단계 상태 모델

상태	의미
MONITORING	명령 또는 수동 조치 완료 · **복구 확인 대기**
RESOLVED	복구 검사 또는 운영자 복구 확인 **완료**

읽기 전용 업무 KPI Probe

조치 **완료 이후** 집계 구간의 성공률 · 실패율 · p95 · 거래 수를 업무 API에서 GET으로 조회합니다. 컨테이너 health와 함께 **2회 연속 통과**해야 RESOLVED로 바뀝니다.

쓸수록 똑똑해지는 플라이휠



confirmed + 복구됨 + 확정 원인 + 성공 조치를 모두 입력한 피드백만 새 사례로 승격해, 지식베이스가 오염되지 않습니다.

AI를 쓰되, 고객 정보는 밖으로 나가지 않게

F5

```
# 원문 로그
payment failed rrn=900101-1234567 card=4111 1111 1111 1111
Authorization: Bearer eyJhbGciOi...  password=p@ss

# 저장 · LLM 입력 · API 응답 (전 구간 동일 적용)
payment failed rrn=[RRN] card=[CARD_OR_ACCOUNT]
Authorization: Bearer [REDACTED]  password=[REDACTED]
```

마스킹 대상

주민번호 · 카드/계좌 · 전화 · 이메일 · 비밀번호 · 토큰 · API 키 · URL 자격증명

LLM OFF 스위치

`LLM_EXTERNAL_ENABLED=false` 한 줄로 외부 호출 차단

감사 가능한 운영

- 승인 · 실행 · 수동 조치마다 **운영자 · 시각 · 방법 · 확인 방식** 기록
- 실행 ID 재사용 · 중복 요청은 기존 결과 반환 (멱등성)
- 감사 스트림 90일 · dead-letter 30일 경과분 Elasticsearch 아카이브
- 화면 출력 HTML 이스케이프, 시나리오 데이터는 `synthetic=true`로 분리
- 비밀값 · 정책 파일은 Git · 이미지에 포함하지 않음

> 보안팀이 “쓰지 말라”가 아니라 **“이렇게 쓰면 된다”**고 말할 수 있는 구조.

관리자 화면에서 장애를 일으키고 복구까지 따라가 봅니다

1. **시나리오 실행** — /admin에서 20개 장애 시나리오 중 하나를 선택, /client 화면도 1초 주기로 따라옴
2. **탐지 · 집약** — Fluent Bit 1초 수집 → 장애 1건 생성, '최근 1분' 강조 · 후속 ERROR는 장애로 만들지 않음
3. **진단 · 추천** — 상세에서 원문 · 최근 로그 · 원인 추정 · 유사 사례 · 추천 Runbook 순위 확인
4. **승인 후 실행** — 대상 선택 → '승인 후 실행' → 스크립트 실행 결과 표시 → 상태 MONITORING
5. **복구 확인 · 이력** — '업무 복구 확인' 통과 시 처리 완료, 다른 세션 화면에도 동기화 · Redis / Worker 처리 로그 추적

시연 포인트 — **로그 수십 줄이 장애 1건**으로 · **추천 근거**가 화면에 · **승인 없이는 아무것도 실행되지 않음** · **복구 확인 전까지는 완료가 아님**

중간발표 이후, “다음 과제”를 실제 기능으로 바꿨습니다

항목	중간발표 시점	결과공유회 시점 (master)
장애 시나리오	6종 감지 테스트	**20종** 시나리오 · 오류 코드 **21종** · Runbook 19종 자동 연결
승인 후 실행	“구현 필요” (Next Step)	**구현 완료** — 승인 API + 서버 8가지 조건 검증 + 중복 실행 방지 + 상태 복원
결과 확인	실행 결과 저장 계획	**MONITORING / RESOLVED 분리**, health + 업무 KPI 2회 확인, 수동 조치 이력
추천 품질	하이브리드 사례 검색	+ **품질 게이트 · 자원 가설 엔진 · 결정론 Fallback**, 반복 분석 재사용
운영 · 보안	개발 데이터 기반 검증	전 구간 **마스킹**, P1~P3 우선순위, 운영 콘솔 실시간 동기화, **EC2 자동 배포**
검증	3-프로세스 기동 검증	자동 검사 **158개 중 157개 통과** (1개는 Docker 환경 필요로 생략), 배포 · 부하 점검 도구

중간발표에서 약속한 “운영자가 고르면 안전하게 실행되는 구조”를 **완성했고**, 복구 검증과 보안까지 넓혔습니다.

기존 자산은 그대로 살리고, 운영 업무는 실질적으로 바꿉니다

기존 자산	JARVIS 연계 방식
애플리케이션 로그	Fluent Bit tail로 **업무 코드 수정 없이** 수집
기존 운영 스크립트	SHA-256 Runbook 등록 → 승인 후 실행
업무 상태 · KPI API	읽기 전용 GET으로 **복구 판정**에 활용
Docker · Compose 환경	정책 기반 Execution Agent가 대행
LLM · 검색 인프라	포트 · 어댑터로 사내 LLM · 저장소 교체

업무	Before	After
인지	로그 검색 도구 수동 확인	최근 10분 장애 목록 · 1분 강조
판단	담당자 경험 · 기억	사례 + 진단 근거로 추천
조치	서버 접속 · 수동 명령	콘솔에서 승인 1회
확인	“재시작 성공 = 복구”	health + 업무 KPI 2회
기록	사후 수기 보고	운영자 · 시각 · 방법 자동 이력

새 시스템을 “하나 더” 들이는 것이 아니라, 이미 있는 **로그 · 스크립트 · 업무 API**를 엮어 장애 대응 흐름을 완성합니다.

네 가지 목표, 계획한 범위를 구현했습니다

목표	달성 결과 (master 기준)	상태
G1 End-to-End 파이프라인	수집 → 집약 → 진단 · 추천 → 승인 → 실행 → 복구 확인 → 학습 **7단계 루프**, Compose **11개 서비스** 한 줄 기동	달성
G2 이력 기반 진단 · 추천	오류 **21종** 분류, Qdrant + ES **하이브리드 사례 검색**, 시나리오 **20개 중 19개** Runbook 연결, LLM 없이도 추천 유지	달성
G3 승인 기반 실행	운영자 승인 + 서버 **8가지 조건** 검증, 분산 잠금 · 실행 기록으로 중복 실행 차단, 수동 조치 이력화	달성
G4 현업 적용 수준	**EC2 자동 배포**, 8GB · 2vCPU 자원 설계, 전 구간 마스킹, 자동 검사 **157 / 158** 통과 실제 거래 KPI 연동 · 로그인/RBAC는 운영 환경 정보가 필요해 4부(스케일업)로 이관	핵심 달성

21종
자동 분류 오류 유형

58개
등록 Runbook 스크립트

157/158
자동 검사 통과

5.5GiB
상시 메모리 상한

어려운 문제를 풀었고, 현장에 바로 놓을 수 있습니다

도전 수준 — CoP에서 해결한 기술 과제

- **비동기 분산 처리** — Redis Streams, 재시도 · dead-letter, Worker 생존 신호
- **하이브리드 AI** — 의미 + 키워드 검색, LLM, 결정론 Fallback, 품질 게이트
- **멱등 실행** — 실행 ID · 분산 잠금 · SQLite journal로 응답 유실에도 재실행 없음
- **실시간 동기화** — 관리자 · 클라이언트 화면 간 상태 공유, 늦은 응답 덮어쓰기 방지
- **자원 제약** — ES · Qdrant · 임베딩 모델을 8GB · 2vCPU 한 대에 배치

실행 가능성 — 현업 적용 조건

- **설치** — `docker compose up -d --build` 한 줄
- **망분리 · 폐쇄망** — 외부 LLM 없이도 결정론 추천으로 동작
- **업무 시스템 무변경** — 로그 tail 수집, 읽기 전용 KPI 조회
- **기존 스크립트 재활용** — 등록만으로 승인 실행 대상 편입
- **검증 도구 내장** — 배포 점검, 10 / 50 / 100건 부하 점검
- **실배포** — master push 시 EC2 자동 배포

난이도는 높게, 도입 장벽은 낮게 — **업무 시스템을 바꾸지 않고, 외부망 없이도, 서버 한 대로** 시작합니다.

개발 데이터 검증을 넘어 운영 데이터 학습으로

PHASE 1 · 현재

단일 서버 PoC

- 개발 · 시나리오 데이터로 핵심 흐름 검증
- 오류 21종 · 시나리오 20종
- 승인 기반 실행 · 복구 검증
- EC2 자동 배포

PHASE 2

운영 데이터 · 실 업무

- 운영 로그를 안전하게 수집 · 학습 · 검증
- 실제 거래 KPI probe 연결
- SSO · RBAC 승인 권한 분리
- 온콜 대시보드 · 알림 연계

PHASE 3

정확도 · 자동화

- 운영 이력 축적, evidence · few-shot 고도화
- 추천 거부 · 수정 이력 재학습
- SAFE · LOW risk 조치부터 단계적 자동 실행
- 사내 LLM 전환

PHASE 4

그룹 공통 플랫폼

- K8s · 멀티 클러스터 Agent
- Worker 수평 확장 · P1 우선 큐
- 관계사 공통 AIOps 허브
- 관계사 간 장애 사례 공유

로드맵은 현재 코드가 이미 분리해 둔 경계 — 포트 · 어댑터, 정책 파일, 원격 Agent, 분리된 큐 — 를 그대로 넓히는 방향입니다.

한 팀의 성과를 조직 전체의 자산으로

CoP 팀
PoC 구축 · 시나리오 20종
운영 가이드 작성

›

운영 조직
담당 서비스 등록
실 업무 KPI 연동

›

전사 운영
공통 장애 지식베이스
표준 대응 절차

›

그룹 관계사
공통 AIOps 허브
관계사 간 사례 공유

① 지식이 확산됩니다

운영자가 확정한 사례가 Qdrant · Elasticsearch에 쌓여 **다른 팀의 장애 판단에도 바로 쓰입니다**. 개인 노하우가 조직 자산이 됩니다.

② 설치가 확산됩니다

Compose 한 줄, 8GB 서버 기준 자원 설계 그대로 **다른 팀이 동일하게 설치**할 수 있습니다.

③ 온보딩이 쉽습니다

새 서비스는 오류 규칙 · 실행 대상 · KPI 정책 **파일 등록만으로 편입**됩니다.

④ 경험이 공유됩니다

운영 · 구조 · 검증 문서와 자동 검사로 이관이 쉽고, **"금융권에서 AI를 안전하게 쓰는 설계"**를 사내에 공유합니다.

바꿔 끼우기 쉬운 구조, 숫자로 증명할 성과

확장 지점	현재 구조
LLM 교체	추천 생성 포트 — OpenAI 호환 / 사내 모델 / 결정론 랭커
저장소 교체	로그 저장 · 사례 검색 포트 — ES · Qdrant 어댑터
대상 추가	실행 대상 · Agent 정책 파일 등록
오류 유형 추가	오류 규칙에 패턴 · 진단 · 조치 후보 등록
업무 검증 추가	business_probe 플러그인 (http_kpi)

도입 후 측정할 핵심 지표

MTTR 장애 발생 → RESOLVED까지 시간 (상태 이력으로 자동 산출)

노이즈 원시 ERROR 대비 생성 장애 수 · 반복 분석 재사용률

채택률 1순위 추천 Runbook이 실제 승인 · 복구로 이어진 비율

재발률 RESOLVED 이후 재발 비율 · 사례 승격 건수

모든 지표는 JARVIS가 이미 저장하는 장애 상태 이력 · 승인 / 실행 기록 · 운영자 피드백에서 바로 계산됩니다.

마무리

사람이 로그를 훑으며 하던 “이거 왜 터졌지?”를 JARVIS가 근거와 함께 먼저 답하고, 운영자는 승인 한 번으로 대응하고, 업무 지표로 복구를 확인합니다.

① 목표달성도
목표 4개 구현 · 중간발표 과제 완료

② 회사성장기여도
기존 자산 연계 · 대응 업무 5단계 개선

③ 임팩트/확산성
사례 지식 · 표준 설치 · 파일 등록 온보딩

④ 산출물완성도
핵심 기능 5개 · 시연 · 157/158 검사

⑤ 난이도 · 실행가능성
분산 · 하이브리드 AI · 폐쇄망 동작

감사합니다 · Q&A